Adquisición de Datos de Voltaje mediante un Sistema Basado en Arduino y Divisor de Tensión con LDR

Ingeniería Mecánica y Electromecánica

Poma Carita Gary Javier
poma.9163997@gmail.com

Docente: Ing. Vidher W. Bustillos D.

Resumen

El presente trabajo describe el diseño e implementación de un sistema de adquisición de datos empleando un microcontrolador Arduino y un divisor de tensión con una fotorresistencia (LDR) como sensor. El objetivo fue medir el voltaje, convertirla en una señal eléctrica proporcional, digitalizarla mediante el conversor analógico-digital del microcontrolador y almacenar los datos en formato CSV para su posterior análisis en Python. Los resultados muestran variaciones coherentes del voltaje respecto a los cambios de intensidad de luz aplicados. Se concluye que la integración hardware-software es una herramienta eficaz para aplicaciones de monitoreo en ingeniería.

Palabras claves: adquisición de datos, Arduino, LDR, divisor de tensión, ADC.

Voltaje data acquisition

Abstract

This paper presents the design and implementation of a data acquisition system using an Arduino-based microcontroller and a voltage divider circuit with an LDR as sensing element. The objective was to measure a real physical variable (light intensity), convert it into a proportional electrical signal, digitize it using the ADC module, and store the data in CSV format for graphical analysis in Python. The developed system demonstrated stable readings and coherent voltage variations according to illumination changes. The integration of hardware and software proves to be an efficient solution for real-time monitoring applications.

Keywords: data acquisition, Arduino, LDR, voltage divider, ADC, monitoring.

Introducción

La adquisición de datos es una herramienta fundamental en ingeniería, ya que permite obtener información de sistemas físicos reales para su análisis y modelado. Los microcontroladores como el Arduino en este caso, integran módulos de conversión analógica-digital que facilitan la digitalización de señales de sensores. En este contexto, la fotorresistencia (LDR) representa un dispositivo cuya resistencia varía en función de la intensidad de luz, permitiendo transformar una variable física en una señal eléctrica medible. El objetivo general de la práctica fue implementar un sistema completo de adquisición, almacenamiento y visualización de datos de voltaje.

Desarrollo de la Investigación

Solución propuesta

Se diseñó un circuito divisor de tensión conformado por una resistencia fija de 1 [kΩ] y un LDR, alimentado con 5 V provenientes del Arduino. El punto medio del divisor se conectó al pin analógico A0. El voltaje de salida se determina mediante la ecuación:

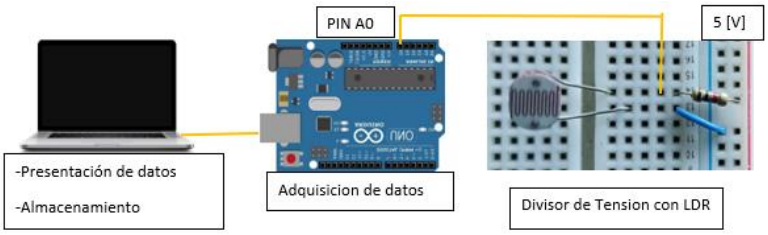
$$Vout = Vcc \cdot \left(\frac{R_{LDR}}{R_{fija} + R_{LDR}} \right)$$

Requerimientos materiales y equipos

- Arduino UNO
- Fotorresistencia (LDR)
- Resistencia fija de 1 kΩ
- Protoboard y cables de conexión
- Computadora
- Arduino IDE
- Python (PySerial, Matplotlib)

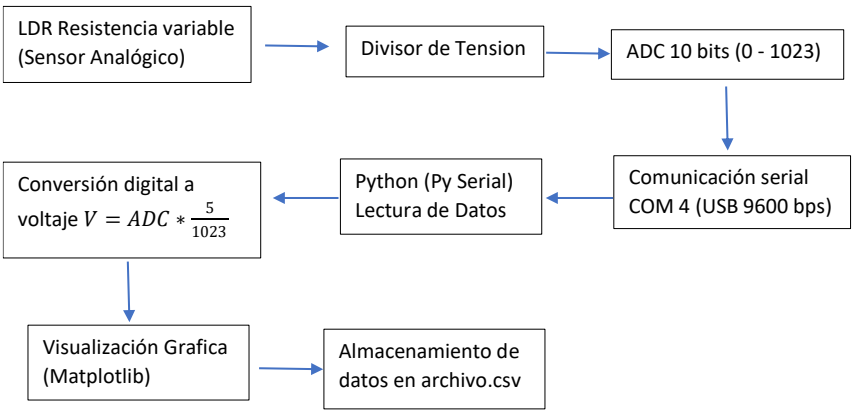
Diagrama en bloques

Figura 1. *Diagrama en bloques.*



Fuente: elaboración propia.

Figura 2. *Diagrama en bloques.*



Fuente: elaboración propia.

Codificación de los programas

IDE Arduino:

Figura 3. *Codificación en el IDE Arduino.*

Ad_datos_Arduino.ino

```
// lectura de LDR
int potPin = A0;
int valor;
void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
}

void loop() {
  valor = analogRead(potPin);
  Serial.println(valor);
  delay(100);
  // put your main code here, to run repeatedly:
}
```

Fuente: elaboración propia.

Codificación en Python:

```
import serial
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from matplotlib.widgets import Button
import csv
import time

# Conexión al Arduino
arduino = serial.Serial(port='COM4', baudrate=9600, timeout=1)

# Archivo CSV 2
archivo = open("ldr4.csv", "w", newline="", encoding="utf-8")
escritor = csv.writer(archivo, delimiter=';')

escritor.writerow(["REGISTRO DE DATOS - SENSOR LDR"])
escritor.writerow(["Fecha:", time.strftime("%d/%m/%Y")])
escritor.writerow(["Hora inicio:", time.strftime("%H:%M:%S")])
escritor.writerow(["Voltaje de referencia:", "5 V"])
escritor.writerow([])
escritor.writerow(["Tiempo (s)", "Voltaje (V)"])

# Variables
tiempo = []
valores = []
inicio = time.time()
```

ADQUISICIÓN DE DATOS DE VOLTAJE

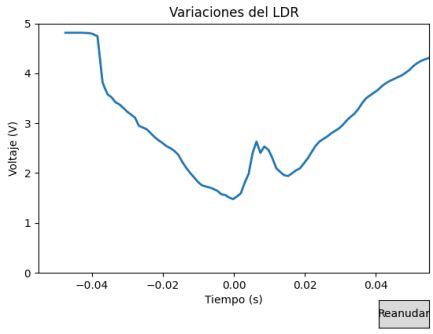
```
pausado = False

# Configuración de la figura
fig, ax = plt.subplots()
plt.subplots_adjust(bottom=0.2)

linea, = ax.plot([], [], lw=2)
ax.set_title("Variaciones del LDR")
ax.set_xlabel("Tiempo (s)")
ax.set_ylabel("Voltaje (V)")
ax.set_ylim(0, 5)

# Función de pausa
def toggle_pausa(event):
    global pausado
    pausado = not pausado
    if pausado:
        boton.label.set_text("Reanudar")
    else:
        boton.label.set_text("Pausar")

# Botón
ax_boton = plt.axes([0.8, 0.05, 0.1, 0.075])
boton = Button(ax_boton, "Pausar")
boton.on_clicked(toggle_pausa)

# Inicialización
def init():
    linea.set_data([], [])
    return linea,

# Actualización de gráfica
def actualizar(frame):
    global pausado

    if pausado:
        return linea,
    dato = arduino.readline().decode().strip()

    if dato:
        try:
            valor = int(dato)
            voltaje = (valor * 5.0) / 1023.0 #conversion de bits a voltaje
            tiempo_actual = round(time.time() - inicio, 2)
```

```
            tiempo.append(tiempo_actual)
            valores.append(round(voltaje, 3))

            escritor.writerow([round(tiempo_actual, 2), round(voltaje, 3)])
            archivo.flush()

            # Mantener últimas 100 muestras
            if len(tiempo) > 100:
                tiempo.pop(0)
                valores.pop(0)

            linea.set_data(tiempo, valores)
            ax.set_xlim(max(0, tiempo[-1] - 10), tiempo[-1])
           # ax.set_xlim(max(0, contador-100), contador)

        except ValueError:
            pass
    return linea,

# Animación
ani = animation.FuncAnimation(fig, actualizar, init_func=init, blit=True,
interval=50)
plt.show()

# Cerrar archivo
archivo.close()
arduino.close()
```

Resultados Obtenidos

Los datos adquiridos fueron almacenados en un archivo CSV donde muestra dos columnas: tiempo (s) y voltaje (V). La gráfica obtenida mostró variaciones de acuerdo con los cambios de iluminación aplicados sobre el LDR, verificando el correcto funcionamiento del sistema.

Figura 4. *Grafica de las Variaciones de Voltaje del LDR obtenida con Python*



Fuente: elaboración propia.

Figura 4. *Datos de tiempo y voltaje obtenidos del archivo CSV.*

REGISTRO DE DATOS - SENSOR LDR	
Fecha:	25/2/2026
Hora inicio:	23:29:33
Voltaje de referencia:	5 V
Tiempo (s)	Voltaje (V)
0.43	0.024
0.49	4.809
0.55	4.809
0.61	4.814
0.67	4.819
0.72	4.814
0.78	4.809
0.84	4.814

0.91	4.814
0.98	4.809
1.04	4.814
1.11	4.814
1.18	4.809
1.24	4.814
1.31	4.814
1.38	4.809
1.44	4.814
1.51	4.814
1.58	4.814
1.64	4.814
1.7	4.814
1.77	4.814
1.83	4.814
1.89	4.814
1.96	4.814
2.02	4.809
2.08	4.809
2.14	4.804
2.21	4.79

2.27	4.765
2.33	4.746
2.39	4.321
2.46	3.822
2.52	3.7
2.59	3.578
2.69	3.519
2.79	3.421
2.89	3.377
2.99	3.309
3.09	3.231
3.19	3.172
3.29	3.113
3.39	2.947
3.49	2.913
3.59	2.879
3.69	2.801
3.79	2.722
3.89	2.659
3.99	2.605
4.09	2.542

5.0	1.755
5.1	1.73
5.2	1.711
5.3	1.681
5.4	1.642
5.5	1.574
5.6	1.559
5.7	1.51
5.8	1.476
5.9	1.53
6.0	1.593
6.1	1.808
6.2	1.984
6.3	2.4
6.4	2.63
6.5	2.405
6.6	2.532
6.71	2.463
6.8	2.312
6.91	2.092
7.0	2.028

Fuente: elaboración propia.

Discusión y conclusiones

Los resultados demuestran la correcta integración entre el sistema físico, el microcontrolador y el software de procesamiento. El método empleado demostró ser adecuado para prácticas académicas de adquisición de datos. Una de las limitaciones identificadas es la resolución del ADC (10 bits) y posibles ruidos eléctricos. Se concluye que el sistema implementado cumple con los objetivos y será de ayuda para los siguientes laboratorios de la materia de métodos numéricos III, y también como una base para aplicaciones más complejas de monitoreo de variables fisicas.

Referencias bibliográficas

Bustillos D., V. W. (2026). *POO_ADQUISICION_1* [Presentación de PowerPoint]. Facultad de Ingeniería, Universidad Mayor de San Andrés.

midudev. (2025, marzo 25). *Curso COMPLETO de Python DESDE CERO para Principiantes 2025* [Video]. YouTube. https://youtu.be/TkN2i-_4N4g

ANEXOS

Figura 5. *Hoja de datos.*

UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE INGENIERÍA

LMEC-237 LABORATORIO DE MÉTODOS NUMÉRICOS Y DISEÑO DE ALGORITMOS III
Ing. Vidher W. Bustillos D.

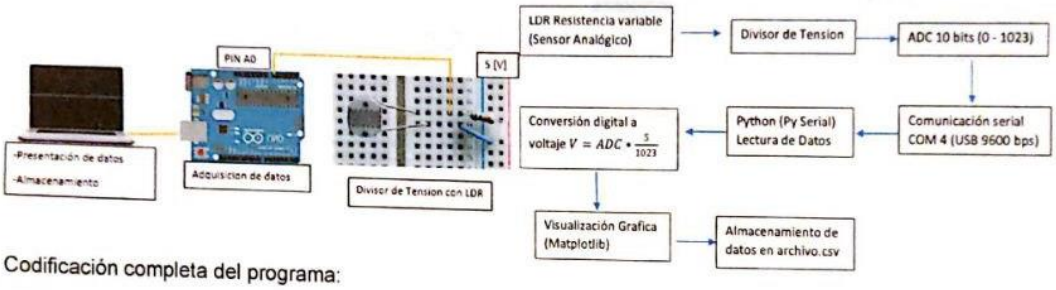
HOJA DE DATOS

LABORATORIO No: 1	FECHA: 25/02/2026					
Adquisición de Datos de Tension		Vo.Bo. Ing. Vidher W. Bustillos D.				
PARTICIPANTES (Apellidos Nombres)		Llenado datos (15)	Presentación de resultados (10)	Funcionamiento Adecuado (30)	Informe en Plataforma (15)	INFORME Fisico (30)	NOTA LABORATORIO (100)
1.- Poma Carita Gary Javier		15	10	30			
2.-							

Nombre completo de la Práctica a desarrollar:
Adquisición de Datos.

Antecedentes del problema a solucionar:
Adquirir datos de un sistema físico real con un microcontrolador, graficar almacenar los datos adquiridos con Python.

Diagrama en bloques o algoritmo de solución del problema:



Codificación completa del programa:

```python
import serial
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from matplotlib.widgets import Button
import csv
import time

# Conexión al Arduino
arduino = serial.Serial(port='COM4', baudrate=9600, timeout=1)

# Archivo CSV 2
archivo = open("ldr4.csv", "w", newline="", encoding="utf-8")
escritor = csv.writer(archivo, delimiter=';')

escritor.writerow(["REGISTRO DE DATOS - SENSOR LDR"])
escritor.writerow(["Fecha:", time.strftime("%d/%m/%Y")])
escritor.writerow(["Hora inicio:", time.strftime("%H:%M:%S")])
escritor.writerow(["Voltaje de referencia:", "5 V"])
escritor.writerow([])
escritor.writerow(["Tiempo (s)", "Voltaje (V)"])

# Variables
tiempo = []
valores = []
inicio = time.time()
pausado = False

# Configuración de la figura
fig, ax = plt.subplots()
plt.subplots_adjust(bottom=0.2)
```

```python
linea, = ax.plot([], [], lw=2)
ax.set_title("Variaciones del LDR")
ax.set_xlabel("Tiempo (s)")
ax.set_ylabel("Voltaje (V)")
ax.set_ylim(0, 5)

# Función de pausa
def toggle_pausa(event):
    global pausado
    pausado = not pausado
    if pausado:
        boton.label.set_text("Reanudar")
    else:
        boton.label.set_text("Pausar")

# Botón
ax_boton = plt.axes([0.8, 0.05, 0.1, 0.075])
boton = Button(ax_boton, "Pausar")
boton.on_clicked(toggle_pausa)

# Inicialización
def init():
    linea.set_data([], [])
    return linea,
```

Fuente: elaboración propia.

Figura 6. *Hoja de datos.*

ADQUISICIÓN DE DATOS DE VOLTAJE

UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE INGENIERÍA

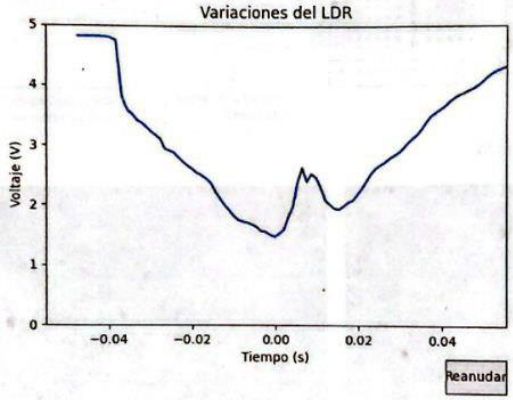
LMEC-237 LABORATORIO DE MÉTODOS NUMÉRICOS Y DISEÑO DE ALGORITMOS III
Ing. Vidher W. Bustillos D.

```python
# Actualización de gráfica
def actualizar(frame):
    global pausado

    if pausado:
        return línea,
    dato = arduino.readline().decode().strip()

    if dato:
        try:
            valor = int(dato)
            voltaje = (valor * 5.0) / 1023.0 #conversion de bits a voltaje
            tiempo_actual = round(time.time() - inicio, 2)

            tiempo.append(tiempo_actual)
            valores.append(round(voltaje, 3))

            escritor.writerow([round(tiempo_actual, 2), round(voltaje, 3)])
            archivo.flush()

            # Mantener ultimas 100 muestras
            if len(tiempo) > 100:
                tiempo.pop(0)
                valores.pop(0)

            línea.set_data(tiempo, valores)
            ax.set_xlim(max(0, tiempo[-1] - 10), tiempo[-1])

        except ValueError:
            pass
    return línea,
```

```python
# Animación
ani = animation.FuncAnimation(fig, actualizar, init_func=init, blit=True, interval=50)
plt.show()

# Cerrar archivo
archivo.close()
arduino.close()
```

Captura de ventanas del funcionamiento adecuado del algoritmo o codificación del programa

- Grafica:



- Archivo.csv:

REGISTRO DE DATOS - SENSOR LDR	
Fecha:	25/2/2026
Hora inicio:	23:29:33
Voltaje de referencia:	5 V
Tiempo (s)	Voltaje (V)
0.43	0.024
0.49	4.809
0.55	4.809
0.61	4.814
0.67	4.819
0.72	4.814
0.78	4.809
0.84	4.814

0.91	4.814
0.98	4.809
1.04	4.814
1.11	4.814
1.18	4.809
1.24	4.814
1.31	4.814
1.38	4.809
1.44	4.814
1.51	4.814
1.58	4.814
1.64	4.814
1.7	4.814
1.77	4.814
1.83	4.814
1.89	4.814
1.96	4.814
2.02	4.809
2.08	4.809
2.14	4.804
2.21	4.79

2.27	4.765
2.33	4.746
2.39	4.321
2.46	3.822
2.52	3.7
2.59	3.578
2.69	3.519
2.79	3.421
2.89	3.377
2.99	3.309
3.09	3.231
3.19	3.172
3.29	3.113
3.39	2.947
3.49	2.913
3.59	2.879
3.69	2.801
3.79	2.722
3.89	2.659
3.99	2.605
4.09	2.542

5.0	1.755
5.1	1.73
5.2	1.711
5.3	1.681
5.4	1.642
5.5	1.574
5.6	1.559
5.7	1.51
5.8	1.476
5.9	1.53
6.0	1.593
6.1	1.808
6.2	1.984
6.3	2.4
6.4	2.63
6.5	2.405
6.6	2.532
6.71	2.463
6.8	2.312
6.91	2.092
7.0	2.028

CS Escaneado con CamScanner

Fuente: elaboración propia.